

Chip Test Cases Documentation

Test Case 1: Register Write/Read Test

Description: Verify basic register write and read operations.

```
#include <stdio.h>
#include "chip_driver.h"

int test_register_operations(void) {
    uint32_t test_value = 0xAA55AA55;
    uint32_t read_value;

    // Write to test register
    if (write_register(REG_TEST_ADDR, test_value) != SUCCESS) {
        printf("Write operation failed\n");
        return TEST_FAIL;
    }

    // Read back the value
    if (read_register(REG_TEST_ADDR, &read_value) != SUCCESS) {
        printf("Read operation failed\n");
        return TEST_FAIL;
    }

    // Verify
    if (read_value != test_value) {
        printf("Value mismatch: wrote 0x%08X, read 0x%08X\n",
            test_value, read_value);
        return TEST_FAIL;
    }

    return TEST_PASS;
}
```

Test Case 2: Memory Pattern Test

Description: Test memory integrity using various patterns.

```
#include <stdio.h>
#include "chip_driver.h"

#define MEM_SIZE 1024 // Size in bytes
#define PATTERN1 0x55555555
#define PATTERN2 0xAAAAAAAA

int test_memory_pattern(void) {
    uint32_t *mem_ptr = (uint32_t *)CHIP_MEM_BASE;
    int i;

    // Write pattern 1
    for (i = 0; i < MEM_SIZE/4; i++) {
        mem_ptr[i] = PATTERN1;
    }

    // Verify pattern 1
    for (i = 0; i < MEM_SIZE/4; i++) {
        if (mem_ptr[i] != PATTERN1) {
```

Chip Test Cases Documentation

```
        printf("Pattern 1 mismatch at offset %d\n", i);
        return TEST_FAIL;
    }
}

// Write pattern 2
for (i = 0; i < MEM_SIZE/4; i++) {
    mem_ptr[i] = PATTERN2;
}

// Verify pattern 2
for (i = 0; i < MEM_SIZE/4; i++) {
    if (mem_ptr[i] != PATTERN2) {
        printf("Pattern 2 mismatch at offset %d\n", i);
        return TEST_FAIL;
    }
}

return TEST_PASS;
}
```

Test Case 3: Interrupt Handler Test

Description: Verify interrupt generation and handling.

```
#include <stdio.h>
#include "chip_driver.h"

volatile int interrupt_received = 0;

void test_interrupt_handler(void) {
    interrupt_received = 1;
}

int test_interrupts(void) {
    // Register interrupt handler
    register_interrupt_handler(TEST_INT_VECTOR, test_interrupt_handler);

    // Enable interrupts
    enable_interrupts();

    // Trigger test interrupt
    generate_test_interrupt();

    // Wait for interrupt (with timeout)
    int timeout = 1000;
    while (!interrupt_received && timeout > 0) {
        delay_ms(1);
        timeout--;
    }

    // Disable interrupts
    disable_interrupts();

    if (!interrupt_received) {
        printf("Interrupt test timed out\n");
        return TEST_FAIL;
    }
}
```

Chip Test Cases Documentation

```
    return TEST_PASS;  
}
```